

TesseraApp

IMPLEMENTATION 2 · PHASE 2 (FINAL)

Software System Implementation & UI Design

The complete, hardened system explained module by module and seam by seam — how the code is organised, how each security-critical path works in code, and what changed since the Phase 1 prototype.

Authors	Robert C. Oliver, Jr. · Travis L. Lester
Course · Advisor	CPSC 69100 Master's Project · Dr. Al-Sharif
Institution	Lewis University
Date · Version	2026 · v3.1

What this report covers

This is the **Implementation 2 (Phase 2, final)** report for TesseraApp — a hybrid, zero-trust Customer Identity & Access Management (CIAM) platform. Where Phase 1 proved the concept end-to-end, Phase 2 documents the *realised* system: every backend subsystem, the security seams, the threat-protection layer, the data layer, the frontend, and how they fit together.

It is written to be read alongside the code and the accompanying implementation video. Every section names the concrete classes involved and, where the logic is critical, shows the passage of code that carries the weight and explains why it is written the way it is. The backend spans **9 REST controllers, 14 service interfaces (16 implementations), 8 configuration classes, 2 servlet filters, 3 security handlers** and a hand-written JDBC identity layer; the sections below walk them in dependency order.

Contents

01 –	Introduction & the complete system	2
02 –	How the code is organised	3
03 –	Two deliberate persistence strategies	4
04 –	The filter chain, headers & CORS	4
05 –	Minting & verifying JWTs	5
06 –	Refresh rotation & reuse detection	6
07 –	Login orchestration — brute-force, anomaly & step-up	7
08 –	Account lifecycle: activation & password-reset email	7
09 –	Authenticator-app MFA (TOTP)	9
10 –	Federated login — find-or-create & link/unlink	10
11 –	Passkeys (WebAuthn): enrollment, usernameless login, admin revoke	11
12 –	Rate limiting & abuse protection	13
13 –	Security dashboard & the audit pipeline	14
14 –	RBAC, capabilities & organization scope	14
15 –	Data layer & schema management	15
16 –	Entity-relationship diagram	16
17 –	Operational hardening & configuration	16
18 –	Frontend implementation (Angular 21)	17
19 –	Build, packaging & deployment	18
20 –	Testing & quality	19
21 –	Testing (continued) — what each suite guards	19

22 –	Changes since Phase 1 · limitations	20
23 –	18.3 — Phase 2 by the numbers	21
24 –	18.4 — Defects found and fixed	22
25 –	UI & implementation video	23
26 –	UI — admin surfaces (continued)	25
27 –	Conclusion	25
28 –	Author Contributions	26
29 –	AI Tool Disclosure	26

01 – Introduction & the complete system

Phase 2 completes the security feature set, adds a full threat-protection layer, consolidates the data layer, and hardens the build — turning the Phase 1 prototype into a coherent, documented, deployed system running live on AWS.

Phase 2 objectives

ID	Objective	Status
P2-1	Complete the auth core — refresh rotation, reuse detection, TOTP MFA, org-scoped admin	Done
P2-2	Consolidate schema management into an idempotent script	Done
P2-3	Comprehensive developer & API documentation	Done
P2-4	Build / packaging via multi-stage Docker	Done
P2-5	Federated login + account link/unlink (Google / GitHub / Microsoft)	Done
P2-6	Automated test suite for the security seams (164 backend / 87 frontend, CI-gated)	Done
P2-7	AWS CI/CD + live cloud deployment (ECS Fargate); externalised config	Done
P2-8	Threat protection (FR-TPF): login-anomaly + step-up + security dashboard; two-tier rate limiting; per-account logout; security headers	Done
P2-9	Experience & access: six-language i18n, command palette, capability-level UI gating	Done
P2-10	Real SMS delivery via Twilio (credentialed & code-complete; awaiting Twilio's A2P 10DLC campaign verification)	Pending
P2-11	Federated go-live — real provider credentials + CloudFront callbacks (Google, GitHub, Microsoft)	Done
P2-12	Distributed security state	Future

The complete system, at a glance

- **Authentication:** registration + email verification, login, logout, password reset, stateless JWT access tokens, server-tracked rotating refresh sessions with reuse detection, and per-account brute-force lockdown.
- **MFA & step-up:** authenticator-app TOTP (QR enrollment, challenge-bound verification, single-use SHA-256 recovery codes); SMS 2FA (Twilio-credentialed and code-complete; live send is pending Twilio's A2P 10DLC campaign verification, so it currently degrades to a logged code); and risk-driven step-up email codes when a sign-in is anomalous.
- **Federation:** OAuth2 / OIDC login (Google, GitHub, Microsoft) with find-or-create; signed-in users **link and unlink** providers via single-use,

provider-bound tickets, with a takeover guard.

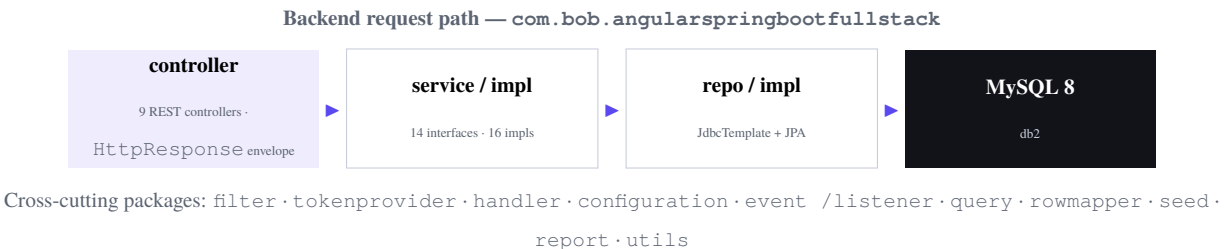
- **Authorization:** seven-role, permission-based RBAC enforced at URL *and* method level, a 1–7 **privilege-tier ladder** guarding role assignment, organization-scoped administration, and a capability catalog that turns every 403 into a specific, non-enumerating message.
- **Threat protection (FR-TPF):** login-anomaly detection with step-up, a two-tier rate limiter (429 + `Retry-After`), per-account lockout, and an admin security dashboard over the audit trail.
- **Administration & audit:** user directory, role reassignment, account-state control, and a services catalog — all written to an immutable audit trail of **18 event types** via an event-driven pipeline.
- **Business domain & analytics:** customers & invoices CRUD, dashboard statistics, XLSX export, and admin-gated billing / analytics rollups.
- **Experience:** runtime six-language i18n (en-es-fr-de-pt-zh), a Cmd/Ctrl-K command palette, and capability-level UI gating.

Architectural summary

Every authentication path — in-house password, TOTP-verified, step-up-verified, or federated OAuth2 — converges on a **single token-issuance seam** (`SessionServiceImpl.issueTokenPair`), so RBAC, MFA policy, session tracking, and audit logging apply identically no matter how a user signs in.

02 – How the code is organised

A strictly layered Spring Boot backend and a standalone-component Angular frontend, deployed as a single container.



Package	Responsibility
controller/	9 REST controllers (User, AdminUser, Customer, Session, Totp, FederatedAuth, SecurityDashboard, Analytics, ServicesCatalog); every response wrapped in the <code>HttpResponse</code> envelope.
service/ · serviceimpl/	Logic behind interfaces — incl. <code>SessionService</code> , <code>TotpService</code> , <code>FederatedIdentityService</code> , <code>LoginRiskService</code> , <code>SecurityDashboardService</code> , <code>OrganizationService</code> , <code>ServicesCatalogService</code> , <code>ImageStorageService</code> .
repo/ · repoimpl/	<code>NamedParameterJdbcTemplate</code> for identity; Spring Data JPA for the business domain.
filter · tokenprovider · handler	<code>CustomAuthFilter</code> , <code>RateLimitFilter</code> , <code>TokenProvider</code> ; custom 401 / 403 handlers and <code>OAuth2LoginSuccessHandler</code> .
configuration	<code>SecurityConfig</code> , <code>OAuth2ClientConfig</code> , <code>JwtSecretGuard</code> , <code>AwsS3Config</code> , <code>TrustedProxyConfigurer</code> , <code>WebMvcConfig</code> .
event/ · listener/	Event-driven audit pipeline (<code>NewUserEvent</code> published, listener persists off the hot path).

Frontend (`tesseraapp/src/app/`): `app.config.ts` (providers), `app.routes.ts` (lazy routes + guards), and folders `features/` (login, home, customers, invoices, security-center, security-overview, users, profile, analytics, billing, services), `service/`, `guard/`, `directive/`, `interceptor/` and `shared/`. No `NgModule` — every component is standalone.

03 – Two deliberate persistence strategies

The codebase uses **two** persistence approaches on purpose, split by risk profile — not by accident of history.

Domain	Strategy	Why
Identity & auth users, roles, sessions, TOTP, org membership, provider links, events	Hand-written, named-parameter SQL via <code>NamedParameterJdbcTemplate</code>	Every query that touches identity is explicit and auditable — no ORM lazy-loading or cascade surprises in the security path.
Business domain customers, invoices, services	Spring Data JPA / Hibernate	Lower-stakes, CRUD-heavy work where the ORM's convenience and derived queries win.

The `User` model is a plain POJO mapped by a hand-written `RowMapper` — it is *not* a JPA entity. SQL lives as constants in the `query/` package (10 query classes); the mapper turns a `ResultSet` row into the domain object:

```
// rowmapper/UserRowMapper.java - explicit, no reflection magic
public User mapRow(ResultSet rs, int rowNum) throws SQLException {
    return User.builder()
        .id(rs.getLong("id"))
        .email(rs.getString("email"))
        .password(rs.getString("password"))           // BCrypt hash - never leaves the service layer
        .enabled(rs.getBoolean("enabled"))
        .isNotLocked(rs.getBoolean("non_locked"))
        .usingMfa(rs.getBoolean("using_mfa"))
        .usingTotp(rs.getBoolean("using_totp"))
        .passwordChangedAt(rs.getTimestamp("password_changed_at"))
        .build();
}
```

```
// repoimpl/UserRepoImpl.java - named parameters, never string concatenation
User user = jdbc.queryForObject(SELECT_USER_BY_EMAIL_QUERY,
    Map.of("email", email), new UserRowMapper());
```

Named parameters make SQL injection structurally impossible in this layer — user input is always bound, never interpolated — and keep the audited security queries readable in one place.

04 – The filter chain, headers & CORS

`SecurityConfig` defines a single `SecurityFilterChain`: CSRF disabled (no server session to forge against), CORS whitelisted, sessions `STATELESS`, authority rules ordered most-specific first, OAuth2 login wired to the federated success handler, a hardened set of response headers, and custom 401 / 403 handlers. `CustomAuthFilter` runs *before* Spring's username/password filter.

```
// configuration/SecurityConfig.java - headers + authority matrix (abridged)
http.headers(h -> h
    .frameOptions(FrameOptionsConfig::deny)           // anti-clickjacking
    .contentTypeOptions(withDefaults())               // nosniff
    .httpStrictTransportSecurity(hsts -> hsts.includeSubDomains(true).maxAgeInSeconds(31536000))
    .contentSecurityPolicy(csp -> csp.policyDirectives( // default-src 'self'; sha256-pinned inline theme script
        "default-src 'self'; script-src 'self' 'sha256-...'; img-src 'self' data: blob: https:; frame-ancestors 'none'"))
```

```

.referrerPolicy(rp -> rp.policy(STRICT_ORIGIN_WHEN_CROSS_ORIGIN))
.permissionsPolicy(pp -> pp.policy("camera=(), microphone=(), geolocation=(), payment=()"))
.csrf(AbstractHttpConfigurer::disable)
.cors(c -> c.configurationSource(corsConfigurationSource()))
.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(auth -> auth
    .requestMatchers(POST, "/user/register", "/user/login").permitAll()
    .requestMatchers(PUBLIC_URLS).permitAll()
    .requestMatchers(PATCH, "/admin/user/*/role/**").hasAnyAuthority("UPDATE:ROLE") // specific FIRST
    .requestMatchers("/admin/**").hasAnyAuthority("UPDATE:USER", "UPDATE:ROLE")
    .requestMatchers("/user/totp/**", "/user/sessions/**").authenticated() // self-service, no staff role
    .requestMatchers(GET, "**").hasAnyAuthority("READ:USER", "READ:CUSTOMER")
    .anyRequest().authenticated())
.oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler)
    .failureHandler((rq, rs, ex) -> rs.sendRedirect(uiAppUrl + "/login?error=federated")))
.addFilterBefore(customAuthFilter, UsernamePasswordAuthenticationFilter.class);

```

Matcher **order is critical**: the narrow admin rules sit above the broad `GET /**` catch-all, so role re-assignment demands `UPDATE:ROLE` and cannot be swallowed by a wider rule. A deliberate **namespace split** keeps the API singular/namespaced (`/user`, `/customer`, `/admin`) while SPA routes are plural/bare (`/customers`, `/billing`) — so permitting an SPA path to load `index.html` can never expose a controller.

Why a forced JSON 401

Enabling `oauth2Login()` installs a redirect entry point that would 302 an expired-token XHR to `/login` — surfacing to the SPA as an opaque CORS failure and defeating the interceptor's silent refresh (which only retries on a real 401). `defaultAuthenticationEntryPointFor(..., AnyRequestMatcher)` overrides it so a missing/expired token always yields a clean JSON 401.

05 – Minting & verifying JWTs

`TokenProvider` mints and verifies HMAC-SHA-512 JWTs. **Access** tokens (30 min) carry the user's authorities and a session family id; **refresh** tokens (5 day) carry only a token id (`jti`) and the family — deliberately no authorities, so `CustomAuthFilter` can never authenticate an API call with a refresh token.

```

// tokenprovider/TokenProvider.java
public String createAccessToken(UserPrincipal user, String family) {
    return JWT.create()
        .withIssuer(TESSERA).withAudience(CUSTOMER_MANAGEMENT_SERVICE)
        .withIssuedAt(new Date()).withSubject(String.valueOf(user.getUser().getId()))
        .withArrayClaim(AUTHORITIES, authoritiesOf(user)) // access token ONLY
        .withClaim(FAMILY, family)
        .withExpiresAt(Date.from(now().plus(30, MINUTES)))
        .sign(HMAC512(secret.getBytes()));
}

```

Validity is more than a signature and expiry. The `passwordChangedAt` guard makes a password change silently invalidate every token issued before it — a token whose `issuedAt` predates the last password change is rejected even though its signature is perfectly valid.

```

// a password change must kill outstanding tokens, everywhere, at once
public boolean isValidToken(Long userId, String token) {
    if (!verifier().verify(token)...) return false; // signature + expiry
    Date issuedAt = decoded.getIssuedAt();
    Date changedAt = userService.getPasswordChangedAt(userId);
}

```

```
return changedAt == null || issuedAt.after(changedAt); // stale-before-change invalid
}
```

Token	Lifetime	Claims	Purpose
access	30 min	sub, authorities, family	Sent on every API request; validated by signature alone (no DB hit).
refresh	5 day	sub, jti, family	Exchanged for a new access token; every use is a tracked DB row.

In production the signing secret is the root of trust for every token; `JwtSecretGuard` (§14) refuses to boot if it is weak or still the placeholder.

06 – Refresh rotation & reuse detection

`SessionServiceImpl` is the single token-issuance seam. `issueTokenPair()` opens a session *family*; `rotate()` exchanges a refresh token for a fresh pair, rotating the `jti` and detecting replay. A refresh presents its `jti`; the row is looked up. If it is already superseded or revoked, that is a replay — the whole family is revoked, a `TOKEN_REUSE_DETECTED` event is audited, and the request is rejected.

```
// serviceimpl/SessionServiceImpl.java - NOT @Transactional (see callout)
RefreshSession session = findByJti(jti);
if (session == null) throw new ApiException("Your session could not be found. Please log in again.");
if (session.isSuperseded() || session.isRevoked()) {
    handleReuse(session); // revoke whole family + publish audit event
    throw new ApiException("This session has been revoked for your security. Please log in again.");
}
jdbc.update(SUPERSEDE_SESSION_QUERY, Map.of("id", session.getId())); // supersede...
String newJti = UUID.randomUUID().toString();
insertSessionRow(userId, session.getFamily(), newJti, request); // ...then insert (fail-closed order)
```

SPA/user/refresh/token
 SessionServiceMySQL 1 **Access token expires**; the interceptor catches the 401 and posts the stored refresh token.
 SPA → /user/refresh/token 2 The handler looks up the row by `jti`.
 SessionService → MySQL 3 **Fresh token**: supersede the old row, insert a new `jti` in the same family (fresh 5-day expiry — sliding sessions), return a new pair.
 SessionService → MySQL → SPA 4 **Replayed token**: the row is already superseded → revoke the entire family and audit `TOKEN_REUSE_DETECTED`.
 SessionService → MySQL 5 **Reject** with a generic message; the user (and the thief) must sign in again.
 SessionService → SPA

A critical design decision — commit before throwing

`rotate()` is deliberately **not** `@Transactional`: the family revocation must *commit* before the rejection throws. Inside a transaction the throw would roll the revocation back — reversing the very containment it had just performed. Supersede-then-insert ordering means a crash mid-way leaves an already-retired token, never two live tokens in one family.

07 – Login orchestration — brute-force, anomaly & step-up

A single sign-in, in `UserController.authenticate()`, passes through four gates before any token is issued. Every failure path is **non-enumerating**: a wrong password, a locked account, and an unknown email return the same shape and message.



Brute-force lockout (M6). Before the password is even checked, `countRecentFailuresByEmail` over a 15-minute sliding window is consulted; at the 5th failure the account is given a persistent lock, so it cannot recover simply by waiting the window out. The rejection is classified as `BRUTE_FORCE_LOCKOUT` in `AuthDiagnosticsLogger` — visible to operators, indistinguishable to the caller.

Anomaly detection (FR-TPF-1). Once the first factor succeeds, `LoginRiskServiceImpl.assess()` compares the request against the account’s own history — reusing the audit trail as the baseline, so no new table is added. Two signals fire, scoped strictly to the one account:

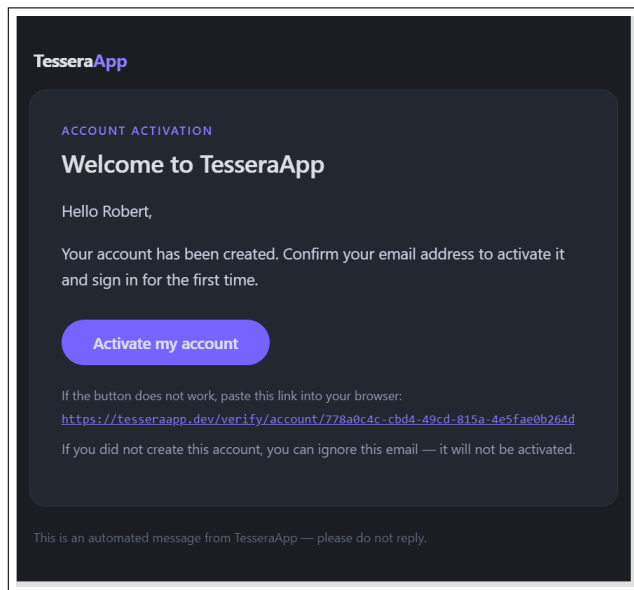
```
// serviceimpl/LoginRiskServiceImpl.java - compare against this account's own baseline only
if (isUsable(currentDevice) && !knownDevices.isEmpty() && !knownDevices.contains(currentDevice))
    reasons.add(LoginRiskReason.NEW_DEVICE); // OS-Browser-Device never seen before
if (currentNetwork != null && !knownNetworks.isEmpty() && !knownNetworks.contains(currentNetwork))
    reasons.add(LoginRiskReason.NEW_NETWORK); // IPv4 /24 or IPv6 /64 prefix - survives DHCP churn
```

Comparison is at **network-prefix** granularity (IPv4 /24, IPv6 routing prefix) so a DHCP renewal on the user’s own sofa is not flagged, while a different ISP, office, or country is. There is deliberately *no* cross-account correlation (an enumeration channel) and no geo-IP dependency. The first-ever sign-in is never flagged.

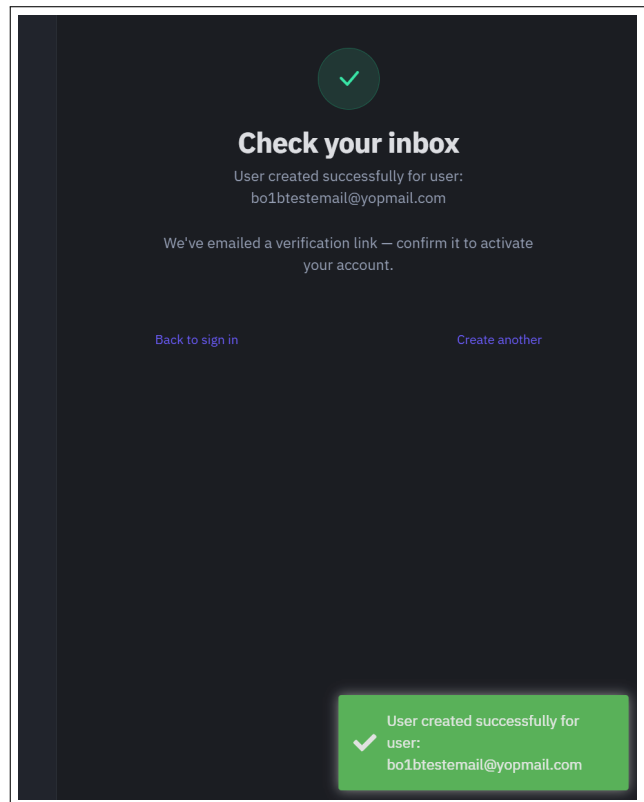
Step-up escalation. When a login is elevated-risk, the second factor is chosen by what the account already has: a confirmed authenticator (TOTP) or SMS 2FA is used as the step-up if present; otherwise a one-time `EMAIL_CODE` is sent. The client cannot tell an anomaly-driven email challenge from an ordinary 2FA prompt — both return `stepUp: true` with no tokens. `recordSuspiciousLogin` audits `SUSPICIOUS_LOGIN` (detail: "a new device → step-up: `EMAIL_CODE`") and emails a security alert. Failure posture is fail-open on the *check* (never break a valid login) but fail-closed on *access*: a fault here logs at `WARN` and the login proceeds without the extra challenge.

08 – Account lifecycle: activation & password-reset email

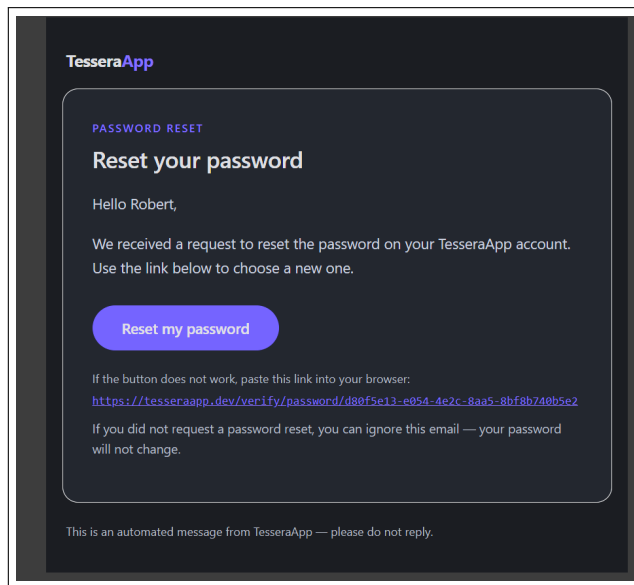
Registration and password reset share one shape: a single-use, expiring token row (`accountverifications` for activation, `resetpasswordverifications` for reset), rendered into an email via `EmailTemplate`, and a link the browser follows back into the app. A new account is created *disabled* and stays that way until the activation link is followed; a reset request never discloses whether the address exists — the confirmation screen is identical either way, consistent with the anti-enumeration posture in §7. Both endpoint groups sit under the rate limiter’s auth tier (§11: `account-verify`, `password-reset`). Now that the project has its own domain, both templates link out through `https://tesseraapp.dev` rather than the CloudFront hostname.



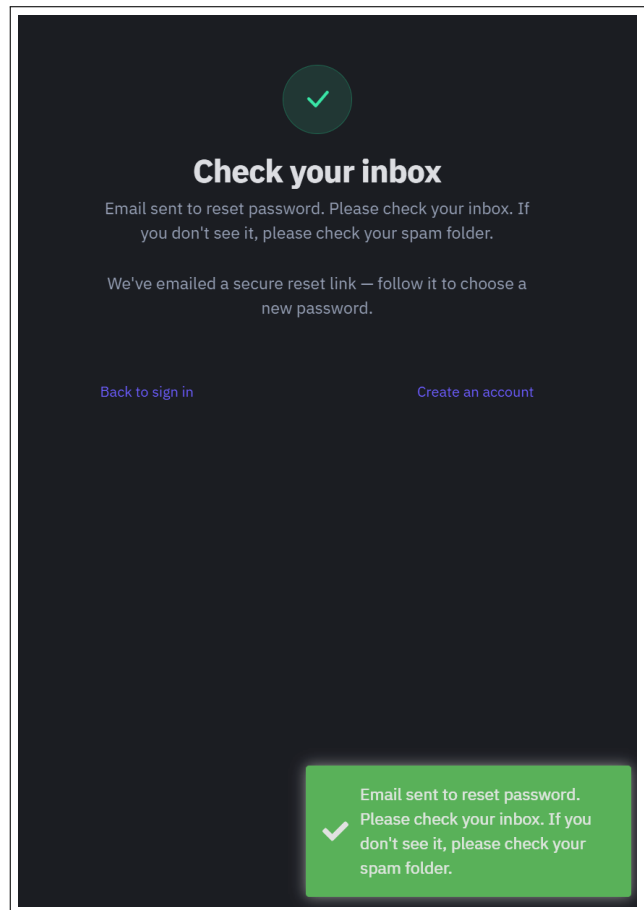
The activation email — the link carries the single-use `accountverifications` token; the account stays disabled until it is followed.



The post-registration confirmation — the same screen and message regardless of whether the address was already taken.



The password-reset email — the link carries a single-use `resetpasswordverifications` token, expiring on the same schedule as account activation.



The post-reset-request confirmation — identical whether or not the account exists, the same anti-enumeration guarantee as registration.

09 – Authenticator-app MFA (TOTP)

`TotpServiceImpl` implements RFC 6238 in-house (`TotpUtils`); only QR rasterization uses ZXing. Enrollment generates a Base32 secret, shows a 240px QR, and requires one correct code to confirm before MFA is switched on — an unconfirmed secret never satisfies anything. Each multi-write step (confirm, disable, issue codes) runs in one transaction so a partial failure cannot strand an account between MFA states.

```
// serviceimpl/TotpServiceImpl.java - login verify is bound to a live, single-use challenge
List<Long> userIds = jdbc.queryForList(SELECT_USER_ID_BY_LIVE_CHALLENGE_QUERY, // expiry lives in the SQL
    Map.of("challenge", challenge == null ? "" : challenge), Long.class);
if (userIds.isEmpty()) throw new ApiException("This sign-in attempt has expired. Please log in again.");
if (!TotpUtils.verifyCode(secret, code)) {
    if (!consumeRecoveryCode(userId, code))
        throw new ApiException("That code didn't match..."); // wrong code does NOT consume the challenge
}
jdbc.update(DELETE_MFA_CHALLENGE_BY_CHALLENGE_QUERY, Map.of("challenge", challenge)); // correct burn once
```

Login verification is **challenge-bound**: a first-factor success mints a short-lived opaque challenge (5 min), and a wrong code keeps the challenge alive for retry until it expires — so possessing the authenticator alone can never skip the password, and expired/forged/consumed challenges are indis-

tinguishable to the caller (NFR-SEC-7). Ten recovery codes are shown once at enrollment, stored only as **SHA-256 hashes**, and burned atomically — the affected-row count of the `UPDATE` is the verdict, so a code cannot be double-spent even under concurrency.

```
// single-use recovery code - the DB does the concurrency control
int consumed = jdbc.update(CONSUME_RECOVERY_CODE_QUERY,
    Map.of("userId", userId, "codeHash", sha256Hex(normalize(code))));
return consumed > 0; // normalize() is case-insensitive and hyphen/space-tolerant
```

Disabling TOTP itself requires a live TOTP or recovery code, so a hijacked browser session cannot quietly strip the second factor. SMS 2FA shares the same challenge machinery and is wired to send for real over Twilio; live delivery is on hold pending Twilio's A2P 10DLC campaign verification, so it still degrades to a logged code in the interim — see §18 and the companion guide.

10 – Federated login — find-or-create & link/unlink

Spring Security's OAuth2 client runs the Authorization Code flow; `OAuth2LoginSuccessHandler` is the token-exchange point. It normalizes per-provider userinfo (Google/Microsoft OIDC `sub`; GitHub numeric `id`), does find-or-create on `(provider, subject)`, applies the *same* account-state and MFA policy as in-house login, then issues our own JWT via `SessionService` — so a federated session appears in the device list like any other.

```
// serviceimpl/FederatedIdentityServiceImpl.java - three-step resolution, one transaction
// 1 · returning user: the (provider, subject) link is the durable key
if (!linkedIds.isEmpty()) return mapToUserDTO(userRepo.get(linkedIds.getFirst()));
// 2 · same verified email already local → link, don't duplicate
if (emailCount > 0) { insertProviderLink(existing.getId(), provider, subject); return ...; }
// 3 · first contact → enabled, passwordless ROLE_USER account
Long id = insertFederatedUser(email, firstName, lastName, imageUrl);
roleRepo.addRoleToUser(id, ROLE_USER.name()); insertProviderLink(id, provider, subject);
```

Account link/unlink lets a signed-in user attach or remove providers from the Security Center. Because a JWT cannot ride a top-level OAuth redirect, `ProviderLinkTicketService` mints a single-use, 5-minute, provider-bound ticket over an authenticated XHR; the ticket grants nothing on its own — it only tells the callback which local account a verified identity should attach to. Two guards make it safe:

```
// linkProviderToUser - the takeover guard (does not reveal WHICH account owns it)
if (!owners.isEmpty() && !owner.equals(userId))
    throw new ApiException("That account is already connected to a different user.");
// unlinkProvider - never strand a federated-only account with no way back in
if (!hasPassword && linkCount <= 1)
    throw new ApiException("This is the only way you can sign in. Set a password first...");
```

Tokens ride in the URL **fragment** (never sent to servers, absent from logs). Only `(provider, subject)` plus a minimal profile is persisted — never a third-party password or token. `OAuth2ClientConfig` (§14) activates each provider only when its credentials are present; `FEDERATED_LOGIN`, `PROVIDER_LINKED` and `PROVIDER_UNLINKED` are audited distinctly.

11 – Passkeys (WebAuthn): enrollment, usernameless login, admin revoke

Built directly on `webauthn4j-core` rather than Spring Security’s session-based WebAuthn module: the app is stateless JWT everywhere else, and that module assumes an `HttpSession` to hold ceremony state. `WebAuthnChallengeStore` fills that role instead — in-memory, single-use, five-minute TTL, structurally identical to `ProviderLinkTicketService` (§9). Passkeys share no tables with TOTP (§8); they are a standalone credential, not a second factor stacked on a password.

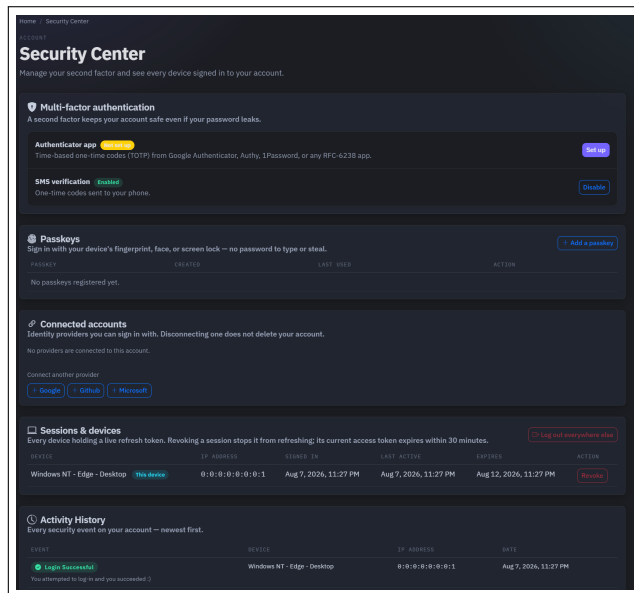
Enrollment is nudged, not forced: an account with no registered passkey sees a one-time, dismissible prompt immediately after its next successful login — accept to enroll on the spot, or defer and add one later from Security Center. The prompt does not reappear once dismissed; it is a suggestion at a natural decision point, not a gate.

```
// PasskeyServiceImpl.java - login: resolve the account from the assertion itself
public AuthResponse finishAuthentication(String credentialJson) {
    AuthenticationData data = webAuthnManager.parseAuthenticationResponseJSON(credentialJson);
    // no userId yet - the challenge was minted anonymously by beginAuthentication()
    if (!challengeStore.redeem(data.getChallengeKey(), AUTHENTICATE))
        throw new ApiException("This sign-in attempt has expired. Please log in again.");
    PasskeyCredential cred = credentialRepo.findByCredentialId(data.getCredentialId())
        .orElseThrow(() -> new ApiException(GENERIC_AUTH_FAILURE)); // same message as an expired challenge
    webAuthnManager.verify(data, authenticationParametersFor(cred)); // checks the live sign_count -
    // clone detection
    credentialRepo.updateSignCount(cred.getId(), data.getSignCount());
    return sessionService.issueTokenPair(cred.getUser(), request); // no LoginRiskService call
}
```

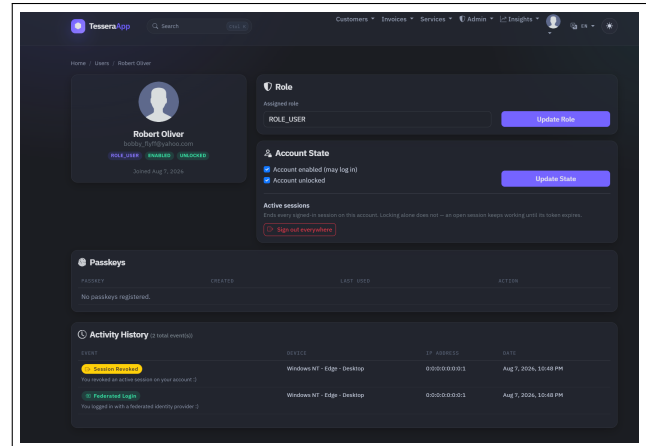
Architectural summary

Login is **usernameless / discoverable**: no identifier is submitted, `navigator.credentials.get()` offers whatever passkey the browser holds for this origin, and the server resolves the account solely from the assertion’s credential id. Because a passkey is already phishing-resistant and bound to one authenticator, a successful passkey login **skips `LoginRiskService` entirely** and mints tokens directly — the same treatment `OAuth2LoginSuccessHandler` already gives federated login (§9): stacking a TOTP or step-up challenge on top of a passkey adds friction, not security.

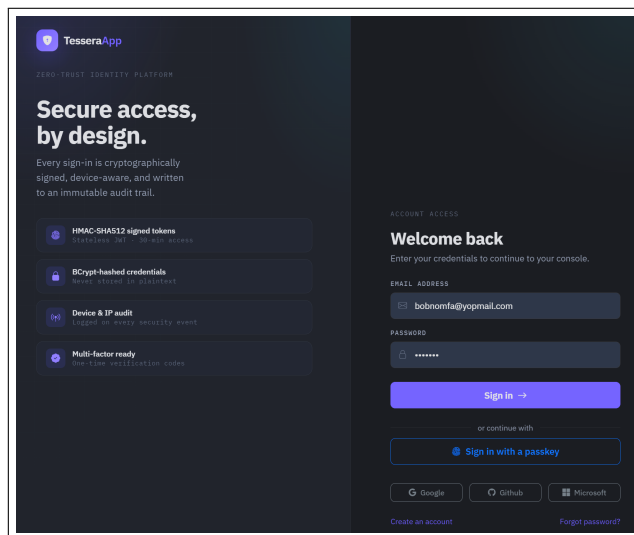
Method	Path	Auth
POST	/user/webauthn/enroll/options	authenticated
POST	/user/webauthn/enroll/complete	authenticated
GET	/user/webauthn/list	authenticated
DELETE	/user/webauthn/{id}	authenticated (self)
POST	/user/verify/webauthn/options	public
POST	/user/verify/webauthn	public
DELETE	/admin/user/{id}/passkeys/{credentialId}	UPDATE:USER
DELETE	/admin/user/{id}/passkeys	UPDATE:USER



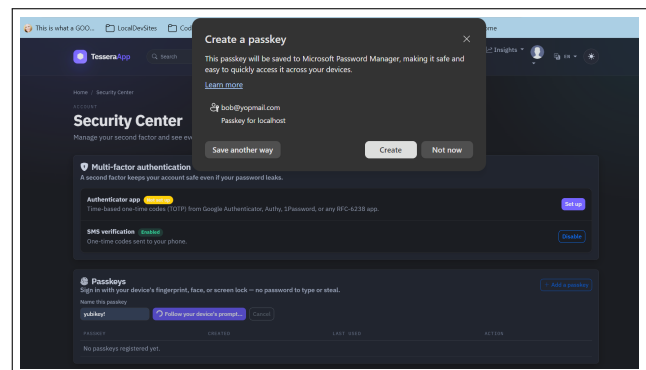
The Security Center's Passkeys card — self-service enroll and revoke, alongside TOTP and SMS.



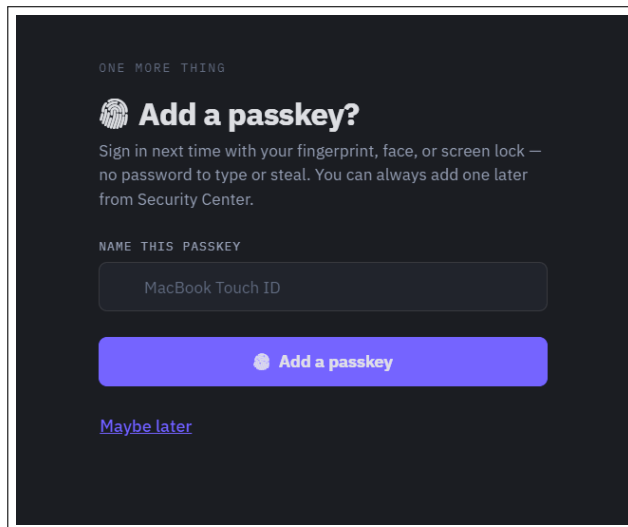
Admin user detail — a staff-only revoke panel scoped to the target user only; the private key never leaves the authenticator, so revoke-and-re-enroll is the only recovery path.



The login form's usernameless entry point — no email typed, the browser's own credential picker takes over.



The platform's own enrollment prompt mid-ceremony — TesseractApp names the credential; the platform (here, Microsoft Password Manager) holds the key.



The post-login nudge — shown once per account with no enrolled passkey; “Maybe later” dismisses it for good and the account falls back to whatever it was using before.

Naming gotcha: the two authenticated endpoints are `enroll/options` / `enroll/complete`, **not** `register/*` — the frontend’s token interceptor withholds the bearer header from any URL with `register` (or `verify`, `login`) as an exact path segment, which is correct for `/user/register` and `/user/verify/webauthn/*` but would silently break an authenticated endpoint that happened to share the word. Every login failure — expired challenge, unknown credential id, failed signature/clone check — returns the same generic message; an unresolved passkey assertion carries no account context worth protecting differently case by case.

12 – Rate limiting & abuse protection

`RateLimitFilter` is a plain servlet filter registered at `@Order(-200)` — *before* Spring Security’s `FilterChainProxy` — so throttling fires before any JWT parsing or authentication work begins. It enforces two tiers with `Bucket4j` token buckets, keyed per client IP.

Tier	Limit	Applies to
Auth	10 req / min / IP	Credential and code endpoints: login, register, refresh, TOTP verify, password-reset, account-verify, verify-code.
Global	200 req / min / IP	Every other path — general API-abuse protection that leaves normal SPA usage untouched.

```
// filter/RateLimitFilter.java - refuse over-limit with a machine-readable 429
ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
if (probe.isConsumed()) { filterChain.doFilter(request, response); return; }
long retryAfter = NANOSCONDS.toSeconds(probe.getNanosToWaitForRefill()) + 1;
response.setStatus(429);
response.setHeader("Retry-After", String.valueOf(retryAfter)); // standard HttpResponse envelope in the body
response.getWriter().write(objectMapper.writeValueAsString(tooManyRequests(retryAfter, path)));
```

The bucket refills greedily (tokens spread across the minute rather than all at reset), and the client IP is resolved from `X-Forwarded-For` at the configured trusted-proxy depth (`TRUSTED_PROXY_COUNT`, so a client

cannot spoof its source IP behind the ALB) so the limiter is correct behind an ALB or nginx. The auth tier keeps brute-force cost prohibitive even *beyond* the per-account lockout of §7 — the two controls are complementary: lockout protects one account, the limiter protects the endpoint.

Known scope

The bucket store is an in-memory `ConcurrentHashMap` — correct for the single-instance deployment. Horizontal scaling needs a shared backend (Bucket4j + Redis / Hazelcast); the filter interface would not change. Same single-instance caveat applies to the brute-force counter and the link-ticket store.

13 – Security dashboard & the audit pipeline

The audit pipeline. Security-relevant actions publish a `NewUserEvent`; `NewUserEventListener` persists the row to `userevents` *off the request's hot path*, so auditing never blocks or fails the operation it records. There are **18** event types in `EventType`, each carrying a human-readable description shown in the Activity Log — including `SUSPICIOUS_LOGIN`, `TOKEN_REUSE_DETECTED`, `RECOVERY_CODE_USED`, `FEDERATED_LOGIN`, `SESSION_REVOKED`, and `PROVIDER_LINKED/UNLINKED`.

The dashboard (FR-TPF-2). `SecurityDashboardServiceImpl` assembles six panels as one read-only `@Transactional` snapshot, so every panel observes the same instant — an admin never sees a suspicious login in the table while the counter above still reads zero.

Panel	What it shows
Event counters	Every reported type, zero-filled so “0 suspicious logins” is a statement, not a missing tile.
Suspicious-login triage	Paged table of anomaly-flagged sign-ins with the signal + step-up detail.
Login-outcome trend	Per-day success / failure / suspicious, gap-filled so a quiet day is a zero bar, not a vanished axis point.
Restricted accounts	Accounts locked by brute-force protection or not yet enabled.
MFA adoption	How many accounts run TOTP vs SMS vs none.
Session activity	Active sessions and how many accounts hold them.

```
// serviceimpl/SecurityDashboardServiceImpl.java - untrusted params clamped; org scope fails closed
int window = Math.clamp(windowDays, 1, 90);           // a hostile ?days=100000 can't force a full scan
if (scoped && organizationIds.isEmpty())
    return SecurityOverview.empty(window);             // org admin with no membership  zeros, never the platform
int suspiciousSize = Math.clamp(suspiciousLoginsSize, 1, 100); // page size bounded like the admin directory
```

Window and page-size arrive as query parameters and are treated as untrusted input to aggregate queries — clamped rather than rejected, and the clamped value is echoed in the pager metadata so a caller who asks for 5000 rows is told what they actually received.

14 – RBAC, capabilities & organization scope

Authorization is **permission-string based** rather than based on coarse role names. A role's permission column (e.g. `READ:USER, UPDATE:CUSTOMER`) is split into `SimpleGrantedAuthority` instances at token time and matched by `hasAnyAuthority(...)`. Enforcement is at two levels that must stay in lockstep: **URL** matchers in `SecurityConfig` and **method** `@PreAuthorize` on the admin controllers — so a future routing

change cannot silently reopen access.

Capability catalog. A 403 is produced in the filter chain, before any controller method is chosen — so the reason has to be derived from the request’s method and path. `CapabilityCatalog` maps that to a specific verb phrase, turning “you don’t have enough permission” into “you don’t have permission to *assign roles* — contact your administrator.” Every phrase names a *capability* only, never whether a record exists (a 403 covers out-of-scope *and* unauthorized), and the phrases are identical to the SPA’s route-guard strings so the user reads one consistent sentence.

```
// constants/CapabilityCatalog.java - ordered most-specific first, like SecurityConfig's matchers
new Rule("PATCH", "/admin/user/*/role", "assign roles"),
new Rule(null, "/admin/security", "view security monitoring"),
new Rule(null, "/admin/analytics", "view billing and analytics"),
new Rule("POST", "/customer/invoice", "create invoices");
```

Organization scope (FR-ORG). `OrganizationServiceImpl` answers “whose data may this admin see?” with a single COUNT over the `userorganizations` membership self-join, and **fails closed** — any error in the scope check denies. `ROLE_ORGANIZATION_ADMIN` is scoped to its active memberships (an empty set means “see nothing”, never “see everything”); `ROLE_ADMIN` and `ROLE_APPLICATION_ADMIN` are unscoped. The rollups behind the Billing/Analytics pages are re-exposed under `/admin/analytics/**` by `AnalyticsController` precisely so a genuine server-side authority check gates them — and the scope restriction is applied *inside the SQL*, because a SUM has already discarded its attribution by the time it is a number.

Privilege tiers & admin session revocation. `RoleType` carries a 1–7 privilege-tier ladder with `canAssign()`; `AdminUserController.requireAssignableTier` refuses any assignment the caller’s own tier does not dominate, blocking *privilege-elevation-by-proxy* (an org admin promoting an in-scope user to an unscoped higher role, then acting through them). Tiers are declared in code — deliberately not read from the database, because seeded role ids have drifted between environments. Alongside it, `DELETE /admin/user/{id}/sessions` gives administrators immediate session revocation: locking an account stops the *next* sign-in; revoking its refresh families ends an intrusion already in progress. A newer, more surgical `DELETE /admin/user/{id}/sessions/{family}` revokes a single device from a Sessions panel on the user-detail page, alongside the existing bulk revoke — both org-scoped, self-target-refused, and audited against the target.

15 – Data layer & schema management

- **Identity / auth tables** (users, roles, user–role, organizations + membership, OAuth provider links, verifications, events + user-events, TOTP credentials + recovery codes, MFA challenges, refresh sessions) are defined in an idempotent `schema.sql` and accessed through `JdbcTemplate`.
- **Business tables** (customer, invoice, services, `invoiceserviceitems`) are JPA-managed by Hibernate (`ddl-auto: update` in development).
- A prior **Flyway** migration set was **removed** after its baseline bookkeeping repeatedly blocked startup; the cumulative schema now lives in one idempotent script.

```
-- schema.sql - safe to run on every boot; never destroys data
CREATE TABLE IF NOT EXISTS RefreshSessions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  family      VARCHAR(36) NOT NULL,      -- one family per login
  jti         VARCHAR(36) NOT NULL UNIQUE, -- current token id in the family
  device      VARCHAR(255), ip_address VARCHAR(45),
```



```

superseded    BOOLEAN DEFAULT FALSE,
revoked       BOOLEAN DEFAULT FALSE,
created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
expires_at    TIMESTAMP NOT NULL,
CONSTRAINT fk_session_user FOREIGN KEY (user_id) REFERENCES Users (id)
);

INSERT INTO Roles (name, permission) VALUES ('ROLE_USER', 'READ:USER,READ:CUSTOMER')
ON DUPLICATE KEY UPDATE permission = VALUES(permission); -- idempotent seed

```

Referential integrity is enforced by foreign keys; single-use keys (verification, reset, recovery) are unique and expiring. A defensive `DemoDataSeeder` seeds the six demo accounts and sample business data on an empty database, so a fresh clone is immediately demonstrable.

Production note

For the cloud target, switch Hibernate to `ddl-auto: validate` and treat `schema.sql` as the single source of truth, so the deployed schema is reviewed and version-controlled rather than inferred at runtime.

Production note

For the cloud target, switch Hibernate to `ddl-auto: validate` and treat `schema.sql` as the single source of truth, so the deployed schema is reviewed and version-controlled rather than inferred at runtime.

16 – Entity-relationship diagram

The relational schema behind §13. The `users` table is the identity hub: every audit row (`userevents`), role assignment (`userroles`), and single-use verification (`accountverifications`, `resetpasswordverifications`, `twofactorverifications`) keys back to it by foreign key. The business domain is a separate cluster — `customer` $\rightarrow$ `invoice` $\rightarrow$ `invoiceserviceitems`, priced against the `services` catalog. The Phase-2 identity tables (TOTP credentials, recovery codes, MFA challenges, refresh sessions) extend this core exactly as the `schema.sql` in §13 defines them.

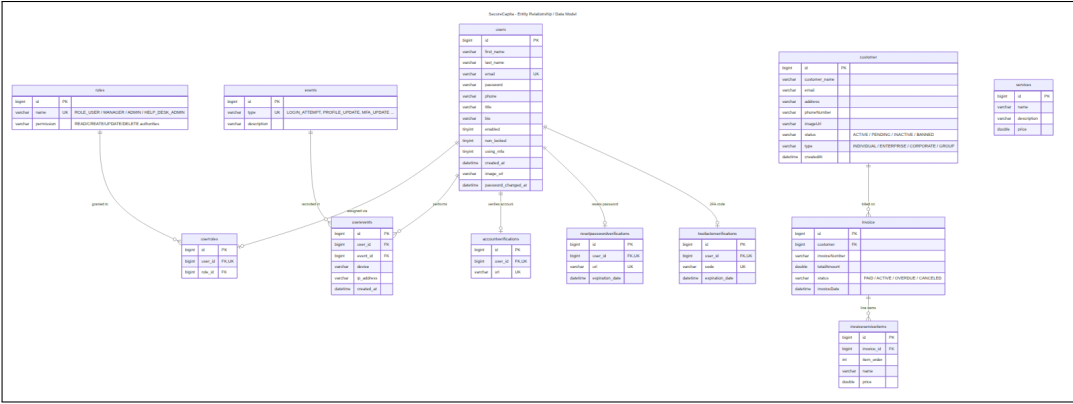


Figure 1. Core relational schema — *users* anchors identity; the customer–invoice–line-item chain forms the business domain; audit, role, and verification tables all key back to *users*.

17 – Operational hardening & configuration

Eight configuration classes turn the app from “runs on my machine” into “boots safely in production or refuses to boot at all.”

Class	What it hardens
JwtSecretGuard	@Profile("prod") + @PostConstruct fail-fast: refuses to start on a missing, placeholder, or <32-char signing secret — closing a total auth-bypass hole. Dev keeps its convenient fallback.
SecurityConfig headers	CSP (with a sha256-pinned inline theme script, not blanket unsafe-inline), HSTS 1yr, X-Frame-Options: DENY, nosniff, Referrer-Policy, Permissions-Policy.
OAuth2ClientConfig	Per-provider registration built in code, activated only when that provider's env credentials are present; a boot-only placeholder keeps the app startable with zero providers and no dead login buttons.
S3ImageStorageService + AwsS3Config	@ConditionalOnProperty (IMAGE_STORAGE_TYPE=s3) swaps local-disk image storage for S3 — required on ephemeral-disk infra (Fargate) where a local volume does not survive a task restart.
TrustedProxyConfig	Configures how far X-Forwarded-For is trusted, so the rate limiter and anomaly detector key on the real client IP behind the ALB — not the proxy, and not an attacker-forged entry.

```
// configuration/JwtSecretGuard.java - a real fail-fast, not a log-and-continue warning
@PostConstruct void verifySecretStrength() {
    if (secret == null || secret.isBlank())         throw new IllegalStateException("JWT_SECRET is not set...");
    if (FORBIDDEN_SECRETS.contains(secret))         throw new IllegalStateException("JWT_SECRET is still the placeholder...");
    if (secret.length() < 32)                       throw new IllegalStateException("JWT_SECRET is too short...");
} // throwing here aborts context refresh - the JVM exits before binding a port
```

Configuration is env-driven across profiles (.env.example, .env.qa, .env.stage); an OWASP dependency-check gate and a build-breaking secret guard mean the pipeline fails on a weak secret or a high-severity CVE before an image ships.

18 – Frontend implementation (Angular 21)

Standalone components bootstrapped from app.config.ts — no NgModule. The critical client-side piece is the **token interceptor**, which attaches the access token and performs a **single-flight silent refresh** on a 401: the first 401 triggers one refresh; concurrent 401s queue on a shared BehaviorSubject and replay once the new token arrives, so a burst of requests never fires a storm of refreshes.

```
// interceptor/token.interceptor.ts - one refresh serves every concurrent 401
if (error.status === 401 && !isRefreshCall(req)) {
    if (!this.refreshing) {
        this.refreshing = true; this.refreshSubject.next(null);
        return this.userService.refreshToken$.pipe(switchMap(res => {
            this.refreshing = false; this.refreshSubject.next(res.data.access_token);
            return next(this.withToken(req, res.data.access_token));
        }));
    }
    return this.refreshSubject.pipe(filter(t => t !== null), take(1), // wait for the single in-flight refresh
        switchMap(t => next(this.withToken(req, t))));
}
```

Capability	Implementation
i18n (6 languages)	Transloco with lazy JSON dictionaries under /assets/i18n; LanguageService lists en-es-fr-de-pt-zh (each labelled in its own language) and persists the choice.
Command palette	Global CommandPaletteComponent (Cmd/Ctrl-K), mounted once, self-gating on authentication; creation/admin entries are gated to the same authorities the navbar uses.
Capability gating	*appHasAuthority removes a control the user can't use; [appRequiresAuthority] disables-and-explains instead of hiding; capabilityGuard is a fail-closed route gate whose denial names the missing capability.
Data state	A DataState<T> wrapper drives explicit LOADING / LOADED / ERROR rendering on every async view; ngx-toastr for non-blocking feedback.

Client guards are a usability layer

Every one of these client gates is a usability aid. Authorization is enforced **server-side** by the filter chain, `@PreAuthorize`, and the capability catalog (§12) — a bypassed frontend guard changes nothing an attacker can reach.

19 – Build, packaging & deployment

Maven builds the backend; the Angular CLI builds the frontend. A **three-stage Dockerfile** compiles the SPA, bakes it into the Spring Boot JAR's static resources, and runs it on a slim JRE with a health check — one self-contained, stateless image.

```
# 1 - build the Angular SPA
FROM node:20 AS web
COPY tesseraapp/ .
RUN npm ci && npm run build -- --configuration production

# 2 - package the JAR with the SPA baked into static/
FROM maven:3.9-eclipse-temurin-21 AS api
COPY pom.xml . ; RUN mvn -q dependency:go-offline
COPY src/ src/
COPY --from=web /web/dist/ src/main/resources/static/
RUN mvn -q -Pprod clean package -DskipTests

# 3 - slim runtime, health-checked
FROM eclipse-temurin:21-jre
COPY --from=api /api/target/*.jar app.jar
HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

The system is **live in production on a full AWS pipeline**: image to ECR, service on ECS Fargate behind an ALB, secrets in Secrets Manager, CloudWatch logs, and managed Aiven MySQL (the JDBC link is now fully TLS-secured with certificate-identity verification — `sslMode=VERIFY_IDENTITY` against a pinned CA on every deployed profile). Database sourcing differs by profile: `dev` defaults to the developer's native/host MySQL but is switchable via `start.sh`'s `DB` flag (`native`, a local Docker container, or `aiven` to inspect the same database production uses); `qa` runs its own dedicated local MySQL Docker container; `prod` is hardcoded to the managed Aiven instance. A **CloudFront distribution fronts the ALB to serve HTTPS**, now reachable at the project's own domain (`https://tesseraapp.dev`, DNS-aliased to the CloudFront distribution with an ACM-issued certificate) in addition to the original `*.cloudfront.net` hostname — the account-verification and password-reset emails already link out through `tesseraapp.dev` (the ALB itself still serves plain HTTP :80). With a real domain now live, the remaining step is pointing `OAuth2_REDIRECT_BASE_URL` and each provider's registered callback at `https://tesseraapp.dev/login/oauth2/code/{provider}` instead of the CloudFront hostname. GCP Cloud Run and Azure App Service pipelines are also built. One-time setup, pipeline stages, and required prod environment are detailed in `aws/RUNBOOK.md` and the companion `final_implementation_guide.md` §3.



20 – Testing & quality

- **Security tooling:** OWASP `dependency-check` in the Maven build (fails on $CVSS \geq 7$); ESLint + Prettier on the frontend; a build-breaking `JwtSecretGuard` that refuses to start on a weak or missing JWT secret.
- **Automated tests (present):** 164 backend (27 suites) and 87 frontend tests, all green and CI-gated, over the security-critical paths — anti-enumeration, brute-force lockout, login-risk assessment, security-dashboard scoping, federated link/unlink, session rotation, TOTP lifecycle, capability guards, and the token interceptor’s concurrent-refresh behavior.
- **Manual verification:** every primary flow exercised end-to-end against a live database — register → verify → login → MFA/step-up → dashboard; admin role reassignment with audit; customer/invoice lifecycle with export; and the password-change-invalidates-all-tokens guarantee.

Testing status

Automated coverage now spans the security seams and gates in CI (**164 backend / 87 frontend, all green**). The remaining test work is end-to-end (Playwright) coverage of the seams in a browser — token rotation + reuse detection, `passwordChangedAt` invalidation, RBAC allow/deny at URL and method level, org-scope, TOTP enroll/verify, and the anti-enumeration guarantee — run in CI so the pipeline fails on a red test.

Seam	Representative test	Level
Refresh rotation	Replay a superseded <code>jti</code> → family revoked + <code>TOKEN_REUSE_DETECTED</code>	Unit
Anti-enumeration	Identical response & timing for known vs unknown email	Integration
Brute-force	5 failures inside the window → persistent lock before the password is checked	Integration
Login risk	New device/network → step-up; first-ever login never flagged	Unit
Org scope	Org admin sees only shared organizations; empty membership zeros	Integration
Federation	Find-or-create idempotent on repeat <code>(provider, subject)</code> ; unlink refuses the last credential	Unit

21 – Testing (continued) — what each suite guards

Phase 1 shipped 14 backend tests and zero frontend specs; Phase 2 ends at **251** ($\times 17.9$). The 22 backend test classes added in Phase 2, and the property each was written to guard:

Test class	Guards
SessionServiceImplTest	Refresh rotation + reuse detection
TotpServiceImplTest	TOTP enrollment and challenge binding
AdminUserControllerOrgScopeTest	Organization-scope enforcement
AdminUserControllerTest	Admin operations, privilege-tier checks, self-target refusal
AnalyticsControllerOrgScopeTest · AnalyticsControllerSecurityTest	Analytics org scoping and method security
LoginRiskServiceImplTest	Risk scoring and step-up selection (both failure directions)
UserControllerBruteForceLockTest	Persistent lockout after 5 failures / 15 min
FederatedIdentityLinkTest · FederatedIdentityUnlinkTest	Link/unlink, duplicate refusal, last-credential protection
SecurityDashboardServiceImplTest	Dashboard aggregates and scoping
CapabilityCatalogTest	Permission-catalog integrity
AuthDiagnosticsLoggerTest	Seven-reason denial classification, server-side only
ErrorDetailScrubberTest	Production error hygiene (no <code>devMessage</code> leak)
NewUserEventListenerTest	A failing audit write can never break authentication
EventServiceImplTest	Audit persistence
RequestUtilsIpAddressTest	X-Forwarded-For parsing at the configured proxy depth
SqlTableCaseConsistencyTest	Every query matches <code>schema.sql</code> 's exact table-name casing
RateLimitFilterTest	Two-tier rate limiting — 429 + <code>Retry-After</code> , per-IP token buckets
OAuth2ClientConfigRedirectUriTest	The pinned <code>https</code> redirect origin holds across all three providers
DatasourceTlsConfigTest	Every deployed profile requires TLS on the database link (<code>sslMode=REQUIRED</code>)
SMSUtilsTest	Real Twilio send path is wired (no longer a no-op); live delivery still gated on Twilio's A2P verification

The 8 frontend spec files cover both guards, the capability directive, the token interceptor's single-flight refresh, authority parsing, the command palette, and the page-size selector, on a reusable harness (`testing/jwt.ts`, `local-storage.ts`, `transloco-stub.ts`).

Reading the test output

The backend run prints a full `Unknown column 'detail' in 'field list'` stack trace. It is **deliberately injected** by `NewUserEventListenerTest` to reproduce a real production incident; the suite is green when it appears — trust `target/surefire-reports`, not the console tail.

22 – Changes since Phase 1 · limitations

What Phase 2 added

- Completed refresh-session **rotation + reuse detection** and the user-facing Security Center (list / revoke devices).
- Added **authenticator-app (TOTP) MFA** with QR enrollment, challenge-bound verify, and single-use recovery codes.
- Completed **OAuth2 federated login** with account **link/unlink** for Google / GitHub / Microsoft — working end-to-end both locally and in the deployed environment, with real provider credentials and registered callbacks now live (HTTPS is already served by CloudFront).
- Built the full **threat-protection layer (FR-TPF)**: login-anomaly detection with step-up, an admin security dashboard, two-tier rate limiting, and per-account brute-force lockout.

- Added **organization-scoped administration**, a **capability catalog** for specific 403s, and admin-gated **billing / analytics** plus a **services catalog**.
- Extended organization scoping from analytics/billing to the shared `/customer/**` surface: `ROLE_ORGANIZATION_ADMIN` now sees only customers and invoices owned by its active organizations, reusing the same `*ForOrganizations` pattern `AnalyticsController` already established. New suite: `CustomerControllerOrgScopeTest` (14 tests) — caught two pre-existing bugs along the way (an immutable-list null check that threw instead of returning false, and a null-valued map building a 500 on a draft invoice).
- Added public `/privacy` and `/terms` pages and gated SMS 2FA opt-in behind explicit consent in the Security Center, linking out to those terms.
- Centralized **password-complexity policy** (`PasswordPolicy`: minimum 8 characters, upper- and lowercase letters, and a digit) so registration, change, and reset agree — previously the reset path enforced only `@NotEmpty`.
- Hardened operations: **security headers** (CSP / HSTS / Referrer-Policy / Permissions-Policy), fail-fast `JwtSecretGuard`, S3 image storage, and env-driven config across profiles.
- Completed the database link's **TLS hardening**: full certificate-identity verification (`sslMode=VERIFY_IDENTITY`) against a pinned CA on every deployed profile, guarded by `DatasourceTlsConfigTest`.
- Added **six-language i18n**, a Cmd/Ctrl-K command palette, and capability-level UI gating.
- **Deployed to production on AWS ECS Fargate** and grew the suite to **164 backend / 87 frontend tests** gating in CI.
- Removed the Flyway migration tool; reconciled the live schema via an idempotent `schema.sql`.

Limitations & future work

Item	Plan	Guide ref
End-to-end tests	Add Playwright coverage of the seams in a browser, gating in CI alongside the unit/integration suite	§4
Distributed security state	Move the rate-limit buckets, brute-force counter, and link-ticket store to Redis for multi-instance scaling	§3
Build reproducibility	<code>start.sh</code> still calls <code>system mvn</code> , not <code>./mvnw</code> — the one unmet Phase-2 roadmap item; a one-line fix	§4
Breach-corpus password check	Check submitted passwords against a breach corpus (e.g. Have I Been Pwned's k-anonymity API) on top of the existing <code>PasswordPolicy</code> complexity rule; deferred because it adds an outbound network dependency to the registration path	§3
Organization self-service	Orgs are seeded/DB-managed today — no UI to create one or move a user between them; separately, scoping keys off the literal <code>ROLE_ORGANIZATION_ADMIN</code> name rather than a capability, so <code>ROLE_HELP_DESK_ADMIN</code> (which also holds <code>UPDATE:USER</code>) reaches <code>/admin/**</code> unscoped	§3

Every remaining item traces to an SRS requirement ID; the exact files to touch, configuration, and acceptance criteria are in the companion `final_implementation_guide.md`.

23 – 18.3 — Phase 2 by the numbers

Measured between the Phase 1 baseline (commit `725c572`, June 19) and the final head, comparing app-relative paths across the `securecapitaapp` → `tesseraapp` workspace rename so only genuine additions count.

Measure	Value
New backend classes	43
New backend test classes	22 (27 suites total)
New frontend source files	18 (+ 8 spec files)
Existing frontend files reworked	103 of 136
Locales	6×495 keys · exact parity, machine-checked
Automated tests	$14 \rightarrow 251$ (164 backend / 87 frontend) · $\times 17.9$

The 43 new backend classes group into five capabilities: risk-based login and step-up (10 — `LoginRiskServiceImpl` and its `context/query/repo` types); the security dashboard (15 — controller, service, repo, and the aggregate models behind its panels); analytics and the services catalog (4 — `AnalyticsController` closed the authorization gap where billing figures were reachable through a customer-scoped endpoint); object storage (4 — `ImageStorageService` behind local and S3 implementations); and hardening/infrastructure (10 — `RateLimitFilter`, `TrustedProxyConfigurer`, `AuthDiagnosticsLogger` classifying every denial into seven server-side reasons with the client response deliberately unchanged, `ErrorDetailScrubber`, `CapabilityCatalog`, `PasswordPolicy`, `ProviderLinkTicketService`, `BrowserErrorPage`, `JacksonConfig`, `EmailTemplate`). **New endpoint groups:** `/admin/analytics/**`, `/admin/security/**`, `/admin/services/**`, `DELETE /admin/user/{id}/sessions`.

Frontend. Of the 18 new source files: two feature screens (`security-overview`, `services-admin`), shared components (command palette, page-size selector, route animations), seven services — including `current-user.service`, which decoupled the navbar from the customer-list response after seventeen templates had been threading identity down purely on the navbar’s behalf — and the authorization primitives (`has-authority` directive, `capability guard`). The bulk of the frontend work was *reworking* the 103 carried-over files: `i18n`, unified pagination, authority-aware rendering, and chart/empty-state treatment.

Roadmap scorecard. Phase 1 §9 committed to eight items; **seven were delivered** — security-critical-path tests, frontend specs, MySQL port exposure, `jjwt` cleanup, prod-profile validation, object storage, general rate limiting — and one was not: `start.sh` still calls the system `mvn` rather than `./mvnw`, a one-line change and the least costly outstanding item. Of Phase 1’s seven carried-forward limitations, **six are closed**; only `.env.example` still shipping a placeholder JWT secret remains, mitigated by `JwtSecretGuard` failing fast under `prod`.

24 – 18.4 — Defects found and fixed

Documented in full as a 20-entry problem log in `documentation/IMPLEMENTATION-HISTORY.md` §4; each entry was identified during real use rather than during review. Highlights:

Defect	Resolution
Login returned 500 for every user	A schema drift made audit inserts fail, and Spring’s synchronous event multicaster propagated the failure into login; the listener now swallows and logs, so a failed audit write can never break authentication
Analytics authorization gap	Billing/analytics data was gated only client-side → dedicated <code>AnalyticsController</code> with <code>@PreAuthorize</code>
Two CORS policies silently disagreed	A hardcoded list in <code>SecurityConfig</code> beat the configurable one; unified into one bean
Admins saw the non-admin view after token refresh	Authority flags were latched at construction rather than read from the current token; <code>canViewBilling</code> moved from a field to a getter
A usable bearer token was logged to the browser console	Removed
Table-name casing differed between environments	Every query aligned to <code>schema.sql</code> ; guarded offline by <code>SqlTableCaseConsistencyTest</code>
CSP broke icons and fonts in production only	Policy corrected
A rejected JWT returned 400 , so silent refresh never fired	Corrected to 401
<code>devMessage</code> leaked in production	<code>ErrorDetailScrubber</code> strips it from error responses
Seeded role ids drifted between databases	Privilege tiers moved into code, deliberately not read from the database
N+1 queries on customer and invoice lists	Fixed with <code>@BatchSize(50)</code> on the invoice and line-item collections — batched IN loads, ~96% fewer SQL round-trips

25 – UI & implementation video

The seams above are enforced server-side, but the user meets them through a dark-first “security console” UI — shipped with dark and light themes behind a persisted one-click toggle (the captures in this section show the light theme): the admin directory below is organization-scoped and every state change on it is audited by the pipeline in §11.

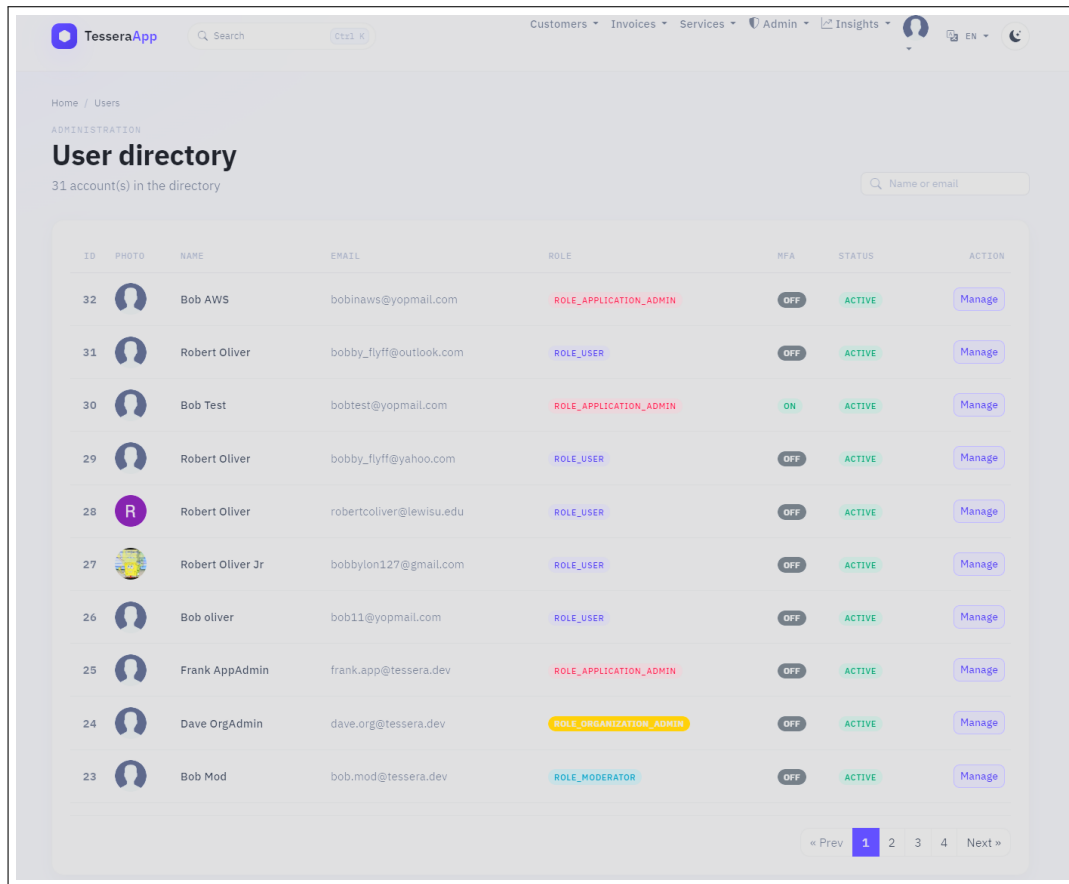


Figure 2. Organization-scoped admin user directory — role, MFA, and account-state, enforced server-side by the security seams in §4–§12.

Implementation video — segment plan

A video walks through the code live. Suggested segments (≈ 10 min total):

Time	Segment	Show
0:00	Tour the layout	Package tree (§2); the two persistence strategies (§3)
1:30	The filter chain	SecurityConfig headers + matrix, CustomAuthFilter (§4)
3:00	Rotation live	Log in, refresh, replay a token → family revoked (§6)
4:30	Login risk	Sign in from a new network → step-up email; dashboard row (§7, §11)
6:00	MFA	Enroll authenticator, challenge-bound verify (§8)
7:30	Federation	Link/unlink a provider; find-or-create on the seam (§9)
9:00	Deploy	Three-stage Docker build; live AWS health check (§16)

Companion documents

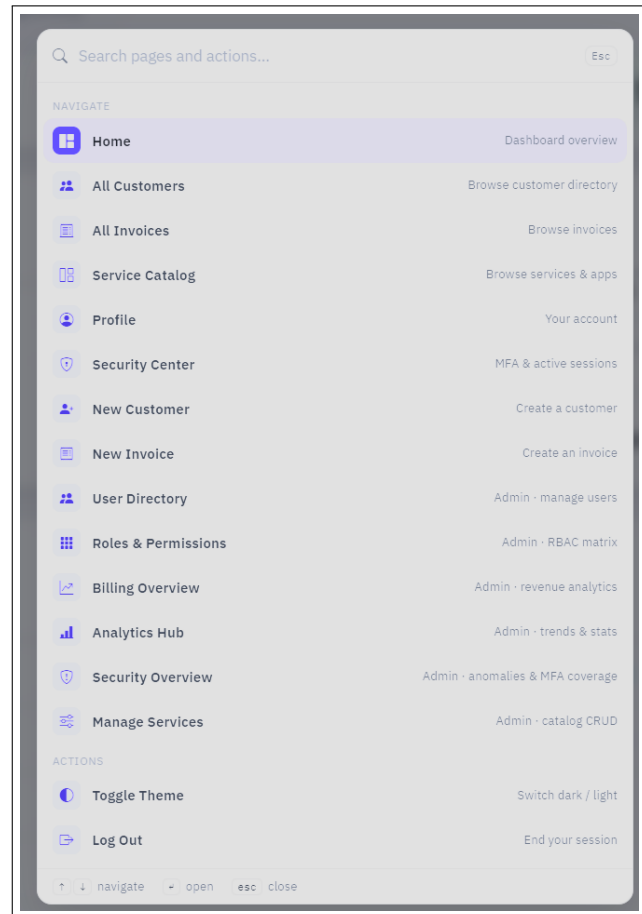
Read alongside the *Final Report / Combined Paper* (problem, literature, SRS, architecture) and the `final_implementation_guide.md` (the remaining-work development plan).

26 – UI — admin surfaces (continued)

Two more Phase-2 surfaces, both rendered only for staff-grade accounts. Catalog administration is deliberately separate from the public browse page: it lists retired entries so they can be reinstated (nothing is ever deleted), and its write endpoints live under `/admin/services/**`, so the authority is enforced server-side regardless of the route guard. The Cmd/Ctrl-K command palette self-filters — creation and admin entries appear only when the signed-in account holds the authority the target route demands (§12, §17).

SERVICES	DESCRIPTION	BASE PRICE	STATUS	ACTIONS
Identity & Access Review	Audit of roles, permissions, and account lifecycle against least-privilege policy.	\$2,400.00	Offered	Edit Retire
Single Sign-On Integration	Connect an existing identity provider (Google, Microsoft Entra, Okta) via OAuth2/OIDC.	\$3,600.00	Offered	Edit Retire
Multi-Factor Rollout	Enrollment campaign and support runbook for authentication-app MFA across an organisation.	\$1,800.00	Offered	Edit Retire
Security Posture Assessment	Review of authentication, session handling, and audit coverage with a prioritised findings report.	\$4,200.00	Offered	Edit Retire
Cloud Migration	Consolidate and migrate an existing deployment to managed cloud infrastructure.	\$7,500.00	Offered	Edit Retire
Database Administration	Schema review, index tuning, backup verification, and restore rehearsal.	\$1,500.00	Offered	Edit Retire
Web Application Development	Custom feature development against the existing angular and Spring boot stack.	\$5,200.00	Offered	Edit Retire
API Integration	Design and build an integration between this platform and a third-party system.	\$2,800.00	Offered	Edit Retire
Compliance Reporting	Evidence pack assembled from the audit log for an external assessor.	\$1,950.00	Offered	Edit Retire
Onboarding & Training	Administrative and end-user training, delivered remotely with recorded sessions.	\$950.00	Offered	Edit Retire

Services-catalog administration — add, edit, retire, reinstate; staff-only via the admin guard and `/admin/services/**`.



The command palette from an admin session — the User Directory, Roles & Permissions, and Billing entries are authority-filtered.

27 – Conclusion

Phase 2 turned the Phase 1 prototype into a **complete, production-deployed CIAM platform**. Every authentication path — password, TOTP-verified, step-up-verified, or federated OAuth2 — now converges on the single token-issuance seam, so RBAC, MFA policy, and audit logging apply identically no matter how a user signs in. Around that core, this phase added the full threat-protection layer (login-anomaly detection with step-up verification, the administrative security dashboard, two-tier rate limiting, and persistent brute-force lockout), authorization in depth (URL + method RBAC, the 1–7 privilege-tier ladder, organization scope, and the capability catalog that turns every 403 into a spe-

cific sentence), and the product surface a real deployment needs: admin-gated billing and analytics, the services catalog with staff-only administration, six-language i18n, and the Cmd/Ctrl-K command palette.

The claim this report supports is intentionally narrow. We do not argue that an in-house identity system beats a commercial one in general — we argue that it can be *built correctly*, and the supporting evidence is operational: the system is **live on AWS ECS Fargate** behind an ALB against managed Aiven MySQL, gated by a CI pipeline that runs **164 backend and 87 frontend tests**, lint, dependency auditing, and a production build on every push. The hybrid stateless/stateful token model — signature-only access verification with server-tracked, rotating refresh sessions and reuse detection — held up under the whole Phase-2 feature load without structural change — strong evidence that the architectural seams were correctly chosen.

The remaining work is enumerated in §18.2: browser-level end-to-end tests, moving security state off the JVM heap for multi-instance scaling, and self-service organization management round out the list. Federated login is complete and fully live for all three providers, with real credentials and registered callbacks at the project’s own domain (<https://tesseraapp.dev>, now live over CloudFront). Each traces to an SRS requirement and has a written plan in the companion `final_implementation_guide.md`. The platform is complete as specified, with the remaining hardening work explicitly enumerated.

In summary

Phase 2 transformed the prototype into a deployed product: a single auditable token core, layered defenses around it, a real business domain behind it, and a test suite that gates every change affecting it.

28 – Author Contributions

R. C. Oliver, Jr. was responsible for the system architecture, the full backend and frontend implementation, all deployment and CI/CD engineering, and authored all six graded project deliverables (proposal/SRS, Implementation 1, Implementation 2, the Final Report, and the accompanying presentation materials), as well as all presentation slides and both recorded demonstration videos. R. C. Oliver, Jr. wrote the great majority of the project’s code end to end.

T. Lester assisted with early thinking on the AWS deployment approach and on the RBAC role design, contributed to the midterm and final presentation and demo materials, helped edit the first and second presentation/demo videos, and helped proofread submissions.

29 – AI Tool Disclosure

We used AI tools, specifically Claude (Anthropic), to assist with drafting, structuring, and formatting this and other course deliverables throughout this project, as well as with code generation in selected implementation areas, including security-related features. All architectural decisions, system design choices, implementation direction, and academic analysis reflect our own thinking and direction. We disclose this use proactively, in accordance with Lewis University’s Academic Integrity policy.